Formally Verified Autoresearch for Analyzing the Expressivity of Multi-Head Attention

Karthik Viswanathan[1,2*] **Plamen Dimitrov**[1] **Stefano Rocca**[1] **Sarayu Sirikonda**[1,3]
[1]Verified Mechanisms [2]MATS Research [3]University of California, Berkeley

Abstract

Multi-head attention is a central component of the transformer architecture, yet how its expressivity depends on the number of heads remains poorly understood. We study a one-layer, attention-only transformer with additive positional embeddings and define the head complexity $H^*(f)$ of a Boolean function f as the minimum number of attention heads required to represent it. We derive an exact linear-fractional normal form for head complexity, prove a general threshold-degree lower bound, and give an explicit ten-bit function for which this bound is strict. For symmetric Boolean functions, we independently show that head complexity equals the number of times the function changes value between consecutive Hamming-weight levels, even when positional embeddings are allowed. This agrees with the concurrent characterization of Hsu and Xu [2026] in a model without positional embeddings. The symmetric lower bound combines our general threshold-degree bound with the classical result of Aspnes et al. [1994]. These results were discovered and formalized by a frontier model through conjecture generation, theorem proving, and machine verification, while human researchers asked the high-level questions and reviewed the generated theorems. We make the Lean formalization and research traces publicly available to support agent-assisted theorem proving and formal verification in mechanistic interpretability. This work provides a proof of concept for using the capabilities of frontier models, grounded by formal verification, to develop an understanding of transformer architectures.

Keywords: mechanistic interpretability, expressivity, formal verification, autonomous research.

1 Introduction

Mechanistic interpretability is an important component of a broader toolkit for AI safety and oversight Bengio et al. [2026]. Theoretical mechanistic interpretability works Elhage et al. [2021, 2022], Geshkovski et al. [2025] have shaped empirical interpretability Olsson et al. [2022], Cunningham et al. [2023], Viswanathan et al. [2026] by providing useful abstractions, and principled starting points for experimentation. However, developing this kind of theory has been demanding, requiring sustained expert effort to work through the underlying mathematics. Recent advances in language models for mathematical reasoning and formal theorem proving Tsoukalas et al. [2026], Alon et al. [2026], Yang et al. [2023] create an opportunity to accelerate the development of rigorous theory for mechanistic interpretability.

Such theory would be especially valuable for multi-head attention [Vaswani et al., 2017], since a more fundamental account of how it transforms the residual stream could support mechanistic interpretability in both models at scale and toy models. In models at scale, a theory of how attention heads transform and combine features could inform theoretically grounded feature-decomposition

*Correspondence to vkarthik095@gmail.com.

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Workshop: Autonomous Machine Learning Research.

tools [Cunningham et al., 2023, Shafran et al., 2026], by relating the features the probes recover to computations that the architecture can implement. For toy models, a theory of multi-head attention can provide a principled basis for interpreting and generalizing empirical findings about how learned computations are distributed across attention heads [Nanda et al., 2023, Piotrowski et al., 2025]. For instance, the problem studied here arose from an empirical finding in a one-layer transformer trained on modular addition Nanda et al. [2023]: parity was only weakly linearly decodable from individual head outputs, but became nearly perfectly decodable when the head outputs were combined. This suggested that parity was implemented as a distributed computation across heads, motivating us to ask whether single-head attention faces a structural expressivity barrier.

Related work Transformer expressivity has been studied through logical characterizations, approximation theory, and analyses of formal languages and Boolean functions [Weiss et al., 2021, Yang and Chiang, 2024, Sälzer et al., 2026, Kajitsuka and Sato, 2024, Hahn, 2020, Kozachinskiy et al., 2026, Hsu, 2026, Köver et al., 2026]. Yu et al. [2026] establish upper and lower bounds on the parameter complexity required for ϵ-approximation as the number of heads varies, while Hsu [2026] establishes lower bounds on the head count for representing n-bit parity with one-layer transformers. Closest to our setting is concurrent work by Hsu and Xu [2026], who also obtain the exact head-count characterization for symmetric functions in a model without positional embeddings. Appendix D provides a detailed comparison with our work.

Despite this progress, deriving general expressivity guarantees for multi-head attention remains technically demanding because tracking even a single layer involves multiple attention head updates and their interactions. Agent-assisted mathematical reasoning can help navigate this analytical complexity, while formal verification provides a rigorous check on the resulting claims. We apply this framework to a concrete expressivity question for Boolean functions:

> For a Boolean function f, what is the minimum number of attention heads needed to represent f in a single attention layer?

2 Autonomous Research Result

Setup We study a one-layer, attention-only transformer based on multi-head attention [Vaswani et al., 2017], followed by a linear threshold readout at the position of a dedicated query token q, as illustrated in Figure 1. For an input $\mathbf{x}$ comprising n bits, let $\mathcal{I} := [n] \cup \{q\}$ denote the set of sequence positions, and let $\mathbf{u}_j(\mathbf{x})$ denote the input representation at position j, obtained by adding its token and positional embeddings. Each head h applies query, key, and value projections given by $W_Q^{(h)}, W_K^{(h)}, W_V^{(h)}$, and the resulting head outputs are linearly combined through an output projection. Appendix E gives the full parameterization and dimensions.

The final output at the query position is

$$\mathbf{h}_q(\mathbf{x}) = \mathbf{u}_q + \sum_{h=1}^{H} W_O^{(h)} \frac{\sum_{j\in\mathcal{I}} \exp\Big(\Big\langle W_K^{(h)}\mathbf{u}_j(\mathbf{x}), W_Q^{(h)}\mathbf{u}_q \Big\rangle\Big) W_V^{(h)}\mathbf{u}_j(\mathbf{x})}{\sum_{j\in\mathcal{I}} \exp\Big(\Big\langle W_K^{(h)}\mathbf{u}_j(\mathbf{x}), W_Q^{(h)}\mathbf{u}_q \Big\rangle\Big)}. \tag{1}$$

We say that f is *exactly representable* with H heads if there exist model parameters of the form specified in Appendix E, and a linear readout $(\mathbf{w}, \tau)$ such that

$$\forall \mathbf{x} \in \{0,1\}^n, \qquad \big[f(\mathbf{x}) = 1 \quad \Longleftrightarrow \quad \langle \mathbf{w}, \mathbf{h}_q(\mathbf{x})\rangle > \tau\big]. \tag{2}$$

Adding positional embeddings is a seemingly modest but consequential extension: they can break permutation invariance and enlarge the representable function class. This allows us to study nonsymmetric Boolean functions beyond the setting considered by Hsu and Xu [2026].

2.1 Characterizing Head Complexity

The *head complexity* of f is the least number of heads in an exact representation:

$$H^*(f) = \min\{H : f \text{ is exactly representable by an } H\text{-head model}\}. \tag{3}$$

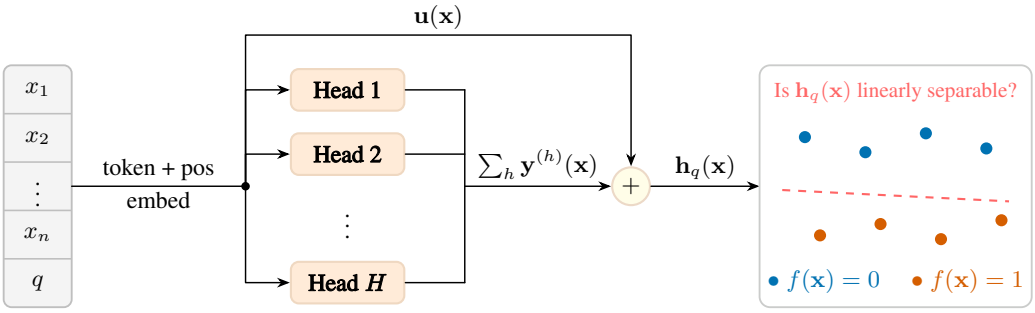


Figure 1: **One-layer attention-only model.** The input consists of n bit tokens followed by a dedicated query token q, which also serves as the readout position. Each position is represented by the sum of a token embedding and a positional embedding, and the resulting embedded sequence is shared by all H heads. The heads update all positions in parallel, but the linear readout uses only position q. At this position, the head outputs are added to the residual $\mathbf{u}_q$, yielding $\mathbf{h}_q(\mathbf{x})$. Thus, f is represented exactly when the inputs labelled 0 and 1 are linearly separable at the readout position $\mathbf{h}_q(\mathbf{x})$, and $H^*(f)$ is the minimum H for which this occurs.

Equations 1 and 2 define $H^*(f)$ through the full set of embeddings and weight matrices, making it difficult to analyze directly. Our first result shows that, after readout, each attention head can be replaced by a single linear-fractional function [Charnes and Cooper, 1962], yielding an equivalent characterization of $H^*(f)$. We now define this one-head building block, which we call an atom and provide the intuition behind this construction in Appendix F.1.

Definition 1 (Linear-fractional atom)**.** For $\gamma, \alpha, \rho_1, \ldots, \rho_n \in \mathbb{R}_{>0}$ and $\eta, \beta, m_1, \ldots, m_n \in \mathbb{R}$, define the corresponding *linear-fractional atom* $\phi : \{0,1\}^n \to \mathbb{R}$ by

$$\phi(\mathbf{x}) = \frac{\eta + \sum_{i=1}^n \rho_i \alpha^{x_i}(m_i + \beta x_i)}{\gamma + \sum_{i=1}^n \rho_i \alpha^{x_i}}, \qquad \mathbf{x} \in \{0,1\}^n. \tag{4}$$

Definition 2 (Linear-fractional complexity)**.** The *linear-fractional complexity* of f, denoted $L_{\mathrm{frac}}(f)$, is the least nonnegative integer H for which there exist linear-fractional atoms $\phi_1, \ldots, \phi_H$ and a constant $c \in \mathbb{R}$ such that, for every $\mathbf{x} \in \{0,1\}^n$, $c + \sum_{h=1}^H \phi_h(\mathbf{x}) > 0 \iff f(\mathbf{x}) = 1$.

Theorem 1 (Linear-fractional normal form)**.** *For every Boolean function f,*

$$H^*(f) = L_{\mathrm{frac}}(f).$$

Theorem 1 reduces head complexity to counting linear-fractional atoms: each head contributes one atom after readout, and each atom can be realized by one head. Appendix F.1 proves this equivalence. The upside of Theorem 1 is that the weight matrices never appear in the rest of the analysis.

Corollary 1 (One head)**.** *$H^*(f) = 0$ if and only if f is constant, and $H^*(f) \le 1$ if and only if f is a linear threshold function: there exist reals $\theta_0, \ldots, \theta_n$ such that, for every $\mathbf{x} \in \{0,1\}^n$, $f(\mathbf{x}) = 1 \iff \theta_0 + \sum_i \theta_i x_i > 0$.*

Appendix F.2 gives the proof. Parity is not a linear threshold function, so Corollary 1 already gives a structural reason why parity was not linearly decodable from single heads in the modular-addition model of Section 1.

We next relate head complexity to threshold degree. Let $\deg_{\pm}(f)$ be the minimum degree of a real polynomial that sign-represents[2] f on the Boolean cube [Minsky and Papert, 1969].

Theorem 2 (Threshold-degree lower bound)**.** *Every Boolean function satisfies*

$$\deg_{\pm}(f) \le H^*(f).$$

By Theorem 1, an H-head score is a constant plus a sum of H linear-fractional atoms. Multiplying by the product of their positive affine denominators preserves its sign pattern on the Boolean cube and yields a polynomial of degree at most H. Appendix F.3 gives the full argument.

[2] A real polynomial P *sign-represents* f if, for every $\mathbf{x} \in \{0,1\}^n$, $P(\mathbf{x}) > 0 \iff f(\mathbf{x}) = 1$.

Strictness of the lower bound. The threshold-degree bound is not always tight. For a ten-bit input $\mathbf{x} = (\mathbf{a}, \mathbf{b}) \in \{0,1\}^{10}$, where $\mathbf{a}, \mathbf{b} \in \{0,1\}^5$, define

$$Q(\mathbf{a}, \mathbf{b}) = \left(\sum_{i=1}^{5}(1-2a_i)\right)\left(\sum_{j=1}^{5}(1-2b_j)\right) - 3\sum_{i=1}^{5}(1-2a_i)(1-2b_i), \tag{5}$$
$$f_{10}(\mathbf{x}) = 1 \iff Q(\mathbf{a}, \mathbf{b}) > 0$$

Theorem 3 (Strict separation from threshold degree)**.** *The function f_{10} satisfies*

$$\deg_{\pm}(f_{10}) = 2 < 3 \le H^*(f_{10}).$$

Appendix F.4 gives the proof. We found autoresearch particularly useful for constructing examples when the bound in Theorem 2 is strict, resulting in the example above.

Constructive upper bounds. We next derive a constructive upper bound for functions that depend on a scalar statistic $t_\lambda(\mathbf{x})$, in terms of the number of distinct values that t_λ takes on the Boolean cube.

Theorem 4 (Weighted-sum interpolation)**.** *Let $\lambda_1, \ldots, \lambda_n > 0$ and define $t_\lambda : \{0,1\}^n \to \mathbb{R}$ by $t_\lambda(\mathbf{x}) = \sum_{i=1}^n \lambda_i x_i$. Suppose f depends on its input only through t_λ, so that $f(\mathbf{x}) = F(t_\lambda(\mathbf{x}))$ for some function $F : \mathbb{R} \to \{0,1\}$. Then*

$$H^*(f) \le |\operatorname{Im}(t_\lambda)| - 1.$$

For $\lambda_i = 2^{i-1}$, we have $|\operatorname{Im}(t_\lambda)| = 2^n$, so $H^*(f) \le 2^n - 1$ for every Boolean function f.

The proof uses $|\operatorname{Im}(t_\lambda)| - 1$ heads to realize the labels assigned by F to the attainable values of t_λ. Appendix F.5 gives the construction.

2.2 Exact Head Complexity for Symmetric Functions

Let $f : \{0,1\}^n \to \{0,1\}$ be symmetric. Equivalently, there is a unique function $F : \{0, \ldots, n\} \to \{0,1\}$ such that, for every $\mathbf{x} \in \{0,1\}^n$, $f(\mathbf{x}) = F(|\mathbf{x}|)$, where $|\mathbf{x}| = \sum_{i=1}^n x_i$ [Paturi, 1992]. We call F the *Hamming-weight profile* of f and define the number of changes in this profile by

$$C(F) = |\{k \in [n] : F(k-1) \ne F(k)\}|. \tag{6}$$

Taking $\lambda_i = 1$ in Theorem 4 gives $H^*(f) \le n$. The following result sharpens this bound to the number of changes in F.

Theorem 5 (Exact symmetric characterization)**.** *For every symmetric Boolean function f with Hamming-weight profile F,*

$$H^*(f) = \deg_{\pm}(f) = C(F).$$

Corollary 2 (Parity)**.** *For every $n \ge 1$, $H^*(\mathrm{PARITY}_n) = n$. This follows from Theorem 5, since parity has n sign changes across Hamming-weight levels.*

Lower bound. Lemma 2.6 of Aspnes et al. [1994] shows that the threshold degree of a symmetric Boolean function equals the number of changes in its univariate Hamming-weight representation, so $\deg_{\pm}(f) = C(F)$. Combining this with Theorem 2 gives $H^*(f) \ge C(F)$.

Upper bound. We construct a degree-$C(F)$ polynomial with sign pattern F by placing one root between each pair of consecutive Hamming-weight levels at which F changes. After dividing this polynomial by a positive polynomial of the same degree, a partial-fraction decomposition yields a representation using $C(F)$ linear-fractional atoms, so Theorem 1 gives $H^*(f) \le C(F)$. Appendix F.6 gives the full construction.

2.3 Machine-Checked Formalization

All results above are formalized and machine-checked in Lean 4. The codebase lays the groundwork for a publicly available Lean environment supporting agent-assisted research in mechanistic interpretability. Section 3 describes the agents' contributions and the human review process.

3 System Design and Meta-Analysis

The theorems in Section 2 were derived by GPT-5.6 SOL from a /goal prompt. The separation[3] in Theorem 3 was obtained through a separate counterexample search initiated by the authors. The Lean formalizations were generated through straightforward prompting, and human researchers subsequently refined the Lean code to make it legible and reusable. This section is based on the anonymized record of the autonomous run, released as `theorems-rollout.jsonl` in the repository accompanying this paper.[4]

3.1 Agent and harness

The theorems in Section 2 were generated by a single GPT-5.6 SOL agent running in a Codex Desktop sandbox. Section 3.2 describes the execution environment. The agent received the problem setup from Section 2 and a comprehensive set of acceptance criteria that provided general research guidance without revealing the expected results. In approximately 9.5 hours, it derived the linear-fractional normal form (Theorem 1), the threshold-degree lower bound (Theorem 2), and the exact symmetric characterization (Theorem 5). The theorem statements, derivations, and supporting constructions were recorded in `answer.md`, the run's primary research output. The agent also explored broader bounds not reported in Section 2. A separate counterexample search subsequently found a 128-bit separation between threshold degree and head complexity, which was later improved to the explicit ten-bit example in Theorem 3.

Defining the problem. Considerable human effort went into specifying the research question precisely and fixing a model against which proposed results could be audited. Appendices G.3 and G.4 reproduce the problem statement and model specification, respectively. Since a closed-form characterization of $H^*(f)$ for arbitrary Boolean functions could not be assumed, the problem statement identified several broader forms of progress, including lower and upper bounds, comparisons with classical complexity measures, and exact results for natural function families.

The theorem-generation prompt. The prompt, adapted from OpenAI [2026], asked the agent to conduct a diversified search across multiple proof strategies. It was submitted through Codex's `/goal` command, which maintains a persistent objective and completion condition across turns [Pathak and Fabbri, 2026]. The prompt instructed the agent to explore genuinely different proof strategies, organize them by their underlying mathematical ideas, and support them with concrete equations or counterexamples. Public web search was disabled so that the agent could not retrieve existing solutions during the run. The run was one-shot: after receiving the initial prompt, the agent completed the search and returned its answer without further human input. Appendix G reproduces the prompt verbatim.

The run used a single `gpt-5.6-sol` process at `xhigh` reasoning effort in Codex Desktop. Session metadata records Codex CLI build `0.148.0-alpha.21`, source `vscode`, a 258,400-token context window, and four context compactions. Although the multi-agent runtime was available, the agent did not spawn any subagents. It used no external connectors and did not invoke Lean during the initial research interval. Lean was introduced only after a subsequent user request because the initial run focused on developing the theory in natural language.

The `/goal` command alone was insufficient. In preliminary runs, supplying only a `/goal` objective led the agent to stop after a short period, make only incremental progress, and mark the goal as complete prematurely. Adapting the persistence and search instructions from OpenAI [2026] was therefore important for sustaining a longer research run.

Counterexample-search setup. The counterexample search used a separate Codex Desktop session with a single GPT-5.6 SOL agent running at `ultra` reasoning effort. Its first turn comprised 32 tool calls completed in under seven minutes. Section 3.4 describes how the search was initiated and refined.

[3] A *separation* is an explicit Boolean function where the threshold degree is strictly smaller than its head complexity, which shows that the lower bound of Theorem 2 is not always tight.

[4] The results had already been obtained in an earlier run. For clarity, this section describes the rollout of a subsequent run conducted without access to the earlier run, although its main observations apply to both.

3.2 Tools

The run used Python 3, NumPy, SciPy, SymPy, and standard shell and Git tools for numerical experiments, symbolic and exact calculations, repository inspection, and testing. Table 1 summarizes the recorded shell activity. Based on command syntax, we classified 373 of the 387 calls and left the remaining 14 unclassified rather than infer their purpose from context. The empirical checks tested the main constructions in `answer.md` using exact arithmetic and exhaustive evaluation on small instances; the other classified calls included additional checks on small Boolean functions and the proposed upper-bound constructions.

Table 1: Recorded shell activity during the theorem-generation run.

Command category	Calls
Python invocations	153
`sed` inspections	70
`rg` searches	40
Runs of empirical checks	24
Numbered file inspections	17
Git queries	14
Other classified calls	55
Unclassified calls	14
Total	387

Sandboxing details. The run executed in a Codex Desktop sandbox on a local Linux virtual machine: a managed permission profile with read access to the whole file system, write access only to the project directory and temporary directories, and no network access. Any action outside these bounds would have required the user's approval, and the rollout records no such request before delivery. Its computations were exact or small-scale numerical checks.

Details on the counterexample search. The counterexample search that reduced the initial 128-bit construction to the ten-bit example in Theorem 3 used Python and exact solvers for small Boolean instances, with public web search available. The formalization was carried out in Lean 4 using Mathlib and Lake, with computationally intensive builds run on an institutional SLURM cluster.

3.3 Research loop

The run proceeded in two stages. The agent spent approximately 9 hours and 38 minutes developing the proofs and producing `answer.md`. It then completed the Lean formalization and committed the results in a further 22 minutes. These timings come from the retained platform event log and include tool latency and idle time.

Table 2: Summary of the autonomous theorem-generation run. Token counts are taken from the last record before the final answer and therefore exclude that turn.

Measure	Summary
Messages	272 intermediate assistant messages and one final answer. The context was compacted four times.
Tool use	The log records 949 tool calls and 960 operations within those calls. The latter comprise 387 shell commands, 307 file edits, 205 continuations of shell sessions, 55 goal checks, and six goal or plan updates.
Token use	The platform reported about 142.4 million cumulative input tokens, of which 140.2 million were cached, and 0.64 million output tokens, including 0.32 million reasoning tokens.
Research process	The agent registered thirteen approach families and used no public web search, external connector, or delegated model call.

Input-token usage is cumulative across model calls, so retained context is counted repeatedly. Of the 142.4 million input tokens recorded, 98.5% were retrieved from the prompt cache.

Exploring proof strategies. During the run, the agent tracked different approaches in `approach_registry.md`. Table 3 groups these approaches by their goals and summarizes their outcomes.

Table 3: High-level summary of the proof families explored during the run.

Goal	Approach	Outcome
Reformulate head complexity	Replace the full matrix parameterization with a scalar description of each head's contribution.	Derived Theorem 1.
Prove upper bounds	Construct representations for functions determined by weighted sums and for symmetric functions.	Derived Theorems 4 and 5, plus bounds not included in the paper.
Prove lower bounds	Compare head complexity with threshold degree and study functions that require more heads.	Derived Theorem 2 and additional lower-bound arguments.
Evaluate candidate approaches	Check proposed identities and constructions on finite instances.	Rejected approaches that failed finite checks and set aside those that still required an unproved lemma.

The agent kept several distinct approaches under consideration rather than committing early to a single line of attack. Patterns observed in small cases were not treated as general results unless the remaining proof steps could be completed. When further exact checks produced counterexamples or showed that a proposed construction could not cover all cases, the corresponding approach was marked as blocked. This process prevented several plausible but unsupported generalizations from entering the final answer.

3.4 Human interventions

Theorem generation. The initial prompt specified the problem and the criteria for a complete answer. From the submission of this prompt to the delivery of `answer.md`, the agent received no further human input, mathematical lemmas, or corrections.

Lean formalization. After the theorem-generation run, the authors asked the agent to commit the artifacts and formalize the core results in Lean. The generated code captured the intended results but was not initially organized for readability or reuse. Human researchers selected the final theorem set, checked that the Lean statements matched the intended claims, and restructured the code to improve its naming, documentation, and use of the existing library.

Counterexample search. The decision to search for an explicit counterexample came from the authors. No such example had emerged during the autonomous theorem-generation runs. Given the one-line goal *find an explicit constructive example when* $\deg_{\pm}(f)$ *is not equal to* $H^*(f)$, the agent's first turn proceeded without further human input and produced a 128-bit example. In subsequent turns, the authors asked the agent to explain candidate examples, find progressively smaller examples with a Lean-friendly proof, and formalize the result in the shared repository. The search ultimately yielded the ten-bit function in Theorem 3.

3.5 Verification

We rely on the Lean formalization to verify the reported theorems. The Lean kernel checks each proof, while a Lean expert checks that its formal statement faithfully captures the meaning, assumptions, and scope of the corresponding natural-language claim. This guards against hallucinated proofs.

Future work could incorporate Lean's `comparator` tool to check untrusted, agent-generated proofs against human-reviewed theorem statements in a sandboxed environment [Lean FRO, 2026].

3.6 Significance of result

The mathematical result treats the number of attention heads as an architectural resource and exactly characterizes the head count required to represent symmetric Boolean functions. It provides a tractable setting for studying architectural expressivity without modeling training dynamics. More generally, it establishes threshold degree as a lower bound on head complexity while showing that this lower bound need not be tight. These results therefore provide a mathematical reference point for asking whether a computation observed across several heads in a trained toy model is required by the architecture or reflects a particular solution found through training.

The broader program is to develop component-level results of this kind into mathematical abstractions for tracking features through attention, feed-forward networks, and multiple layers. Such abstractions could provide principled starting points for empirical studies and, eventually, inform feature-discovery tools for larger transformers. Many current feature-decomposition methods, including sparse autoencoders, learn features from model activations without constraining those features by the computations that the transformer's architecture can implement [Cunningham et al., 2023, Shafran et al., 2026].

An architecture-aware probe could instead ask how many attention heads are required to construct a given feature in a single attention-layer update. The linear-fractional normal form in Theorem 1, which characterizes the contribution realizable by one attention head, provides a theoretical starting point for developing such a probe. The present result is an initial step: it gives exact statements for a one-layer attention only transformer rather than demonstrating transfer to trained models at scale.

Together, the mathematical results and their Lean formalization give other mechanistic interpretability researchers a concrete artifact to reuse and extend. Although the current development is limited to the one-layer Boolean setting studied here, it lays the groundwork for a reusable formal environment in which related questions about transformer components can be stated and verified.

3.7 Interpretability

During the autonomous theorem-generation run, the agent maintained a registry of approaches and their status, and executable scripts for computational checks. The platform recorded prompts, assistant messages and tool calls that can be useful when we want to audit the autoresearch run. Together, these records allow the research trajectory to be reconstructed, including why some approaches were completed or abandoned.

3.8 Limitations

Auditability of autonomous research. The volume of reasoning and results produced during long autonomous runs makes their progress difficult to inspect and steer. Determining which approaches advanced, failed, or were abandoned requires substantial review. Tools that summarize research branches and track the status of claims could reduce this burden and help researchers identify where human guidance would be most useful.

Repetition and premature claims of progress. Extended runs can produce repetitive updates and apparent progress that does not survive scrutiny. The retained rollout contains prolonged monitoring of stalled searches, an incorrect explanation repeated before correction, and a proposed new bound later removed because an existing bound was stronger. These episodes added work without a corresponding improvement in the final result. A monitor that tracks new evidence, flags repetitive or unsupported claims, and recommends redirection or stopping could help guide the search. Whether such monitoring improves research outcomes remains untested.

Together, these limitations motivate treating autoresearch itself as an object of scientific study. Controlled comparisons of prompts, tools, monitoring, human steering, and stopping rules are needed to determine which workflows produce reliable results and how much computational and human effort they require.

4 Broader Impact

Societal benefits and risks. This work contributes to a longer-term effort to make transformer behavior more understandable and amenable to oversight. Precise results about architectural expressivity can provide foundations for interpretability methods, which may eventually help researchers identify and investigate undesirable model behavior by designing theoretically grounded feature-decomposition tools. These benefits remain prospective: our theorems concern exact representation in a one-layer attention only model and do not establish safety guarantees for deployed systems. Furthermore, the research process in this work serves as a proof of concept for mechanistic interpretability researchers who want to pursue mathematically demanding questions but lack ready access to collaborators with expertise in mathematical research or formal theorem proving.

Scientific judgment. Autonomous theorem discovery leaves a central role for human scientific judgment, both in asking informative questions and in assessing the resulting claims. Since the overarching question is broad and has no obvious route to a complete answer, researchers must choose tractable idealizations that isolate meaningful aspects of transformer computation that can inform empirical interpretability. While the state of the art agents are good at solving well defined problems, it remains for the researcher to ask the high level questions and guide the autoresearch runs.

Understanding failures and steering autoresearch. We need to understand its failure modes and how to steer it, especially for open-ended theoretical questions, where no obvious scalar metric captures scientific progress. We propose adapting Inspect [UK AI Security Institute, 2024] and Inspect Scout [Dubois et al., 2026] to audit research runs through their logs. A monitoring agent built on this infrastructure could track verified intermediate results and flag repeated attempts that do not contribute to research progress.

A Reproducibility Statement

All results in this work are accompanied by a public repository available at `https://github.com/VerifiedMechanisms/automlr`. The natural-language theorem statements and proofs are collected in the top-level `theorems/` directory. The corresponding machine-checked Lean 4 development is contained in `formalization/`. The theorem-generation and counterexample-search records are released in `system-design/rollouts/`. Build and proof-integrity checks are documented in `formalization/README.md` and `formalization/BUILDING.md`.

The repository is self contained and no external datasets are required to reproduce the results. The authors did the 9.5 hour autoresearch run using a ChatGPT Pro plan. The machine-checked results in this work are done using Lean 4.31.0 and Mathlib 4.31.0. Refer to the `README.md` in the accompanying repository for detailed instructions.

B Disclosure Statement

Agent(s) / model(s) used

All theorems and their accompanying Lean formalizations were generated with GPT-5.6 SOL in Codex Desktop. Codex also produced the initial draft of the technical paper and assisted with sentence-level revisions. The human authors refined the sections presenting the autonomous research results and wrote the system-design section.

- [✓] A human author has curated this work and verified every claim, figure, methodology detail, and reference.
- [✓] A human author takes full accountability for the submission as a human-authored, human-verified artifact.

Acknowledgments and Disclosure of Funding

This work was supported in part by a Rapid Grant from BlueDot Impact. The grant funded AI tool subscriptions used during the Lean formalization.

C References

Noga Alon, Thomas F. Bloom, W. T. Gowers, Daniel Litt, Will Sawin, Arul Shankar, Jacob Tsimerman, Victor Wang, and Melanie Matchett Wood. Remarks on the disproof of the unit distance conjecture, 2026. URL https://arxiv.org/abs/2605.20695.

James Aspnes, Richard Beigel, Merrick Furst, and Steven Rudich. The expressive power of voting polynomials. *Combinatorica*, 14(2):135–148, 1994. doi: 10.1007/BF01215346. URL https://doi.org/10.1007/BF01215346.

Yoshua Bengio, Stephen Clare, Carina Prunkl, Malcolm Murray, Maksym Andriushchenko, Ben Bucknall, Rishi Bommasani, Stephen Casper, Tom Davidson, Raymond Douglas, David Duvenaud, Philip Fox, Usman Gohar, Rose Hadshar, Anson Ho, Tiancheng Hu, Cameron Jones, Sayash Kapoor, Atoosa Kasirzadeh, Sam Manning, Nestor Maslej, Vasilios Mavroudis, Conor McGlynn, Richard Moulange, Jessica Newman, Kwan Yee Ng, Patricia Paskov, Shalaleh Rismani, Girish Sastry, Elizabeth Seger, Scott Singer, Charlotte Stix, Lucia Velasco, Nicole Wheeler, Daron Acemoglu, Vincent Conitzer, Thomas G. Dietterich, Edward W. Felten, Fredrik Heintz, Geoffrey Hinton, Nick Jennings, Susan Leavy, Teresa Ludermir, Vidushi Marda, Helen Margetts, John McDermid, Jane Munga, Arvind Narayanan, Alondra Nelson, Clara Neppel, Sarvapali D. Ramchurn, Stuart Russell, Marietje Schaake, Bernhard Schölkopf, Alvaro Soto, Lee Tiedrich, Gaël Varoquaux, Andrew Yao, Ya-Qin Zhang, Leandro Angelo Aguirre, Olubunmi Ajala, Fahad Albalawi, Noora AlMalek, Christian Busch, Jonathan Collas, André Carlos Ponce de Leon Ferreira de Carvalho, Amandeep Gill, Ahmet Halit Hatip, Juha Heikkilä, Chris Johnson, Gill Jolly, Ziv Katzir, Mary N. Kerema, Hiroaki Kitano, Antonio Krüger, Kyoung Mu Lee, José Ramón López Portillo, Aoife McLysaght, Olexii Molchanovskyi, Andrea Monti, Mona Nemer, Nuria Oliver, Raquel Pezoa, Audrey Plonk, Balaraman Ravindran, Hammam Riza, Crystal Rugege, Haroon Sheikh, Denise Wong, Yi Zeng, Liming Zhu, Daniel Privitera, and Sören Mindermann. International AI safety report 2026. Technical Report DSIT 2026/001, Department for Science, Innovation and Technology, 2026. URL https://internationalaisafetyreport.org.

A. Charnes and W. W. Cooper. Programming with linear fractional functionals. *Naval Research Logistics Quarterly*, 9(3–4):181–186, 1962. doi: 10.1002/nav.3800090303. URL https://doi.org/10.1002/nav.3800090303.

Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models, 2023. URL https://arxiv.org/abs/2309.08600.

Magda Dubois, Ekin Zorer, Maia Hamin, Joe Skinner, Alexandra Souly, Jerome Wynne, Harry Coppock, Lucas Sato, Sayash Kapoor, Sunishchal Dev, Keno Juchems, Kimberly Mai, Timo Flesch, Lennart Luettgau, Charles Teague, Eric Patey, JJ Allaire, Lorenzo Pacchiardi, Jose Hernandez-Orallo, and Cozmin Ududec. Seven simple steps for log analysis in AI systems, 2026. URL https://arxiv.org/abs/2604.09563.

Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Nova DasSarma, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021. https://transformer-circuits.pub/2021/framework/index.html.

Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, Roger Grosse, Sam McCandlish, Jared Kaplan, Dario Amodei, Martin Wattenberg, and Christopher Olah. Toy models of superposition, 2022. URL https://arxiv.org/abs/2209.10652.

Borjan Geshkovski, Cyril Letrouit, Yury Polyanskiy, and Philippe Rigollet. A mathematical perspective on transformers, 2025. URL https://arxiv.org/abs/2312.10794.

Michael Hahn. Theoretical limitations of self-attention in neural sequence models. *Transactions of the Association for Computational Linguistics*, 8:156–171, 2020. doi: 10.1162/tacl_a_00306. URL https://aclanthology.org/2020.tacl-1.11/.

Daniel Hsu. Lower bounds for one-layer transformers that compute parity, 2026. URL https://arxiv.org/abs/2605.12171.

Daniel Hsu and Mingyue Xu. Attention-based representations for multi-task computation, 2026. URL https://arxiv.org/abs/2608.04243.

Tokio Kajitsuka and Issei Sato. Are transformers with one layer self-attention using low-rank weight matrices universal approximators? In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=nJnky5K944.

Blanka Köver, Alexandra Butoi, Anej Svete, Michael Hahn, and Ryan Cotterell. Understanding the parameter space geometry of transformers encoding boolean functions, 2026. URL https://arxiv.org/abs/2606.08768. Accepted at ICML 2026.

Alexander Kozachinskiy, Tomasz Steifer, and Przemysław Wałęga. Parity, sensitivity, and transformers. *arXiv preprint arXiv:2602.05896*, 2026. doi: 10.48550/arXiv.2602.05896. URL https://arxiv.org/abs/2602.05896.

Lean FRO. Comparator: A trustworthy judge for Lean proofs. Open-source software, 2026. URL https://github.com/leanprover/comparator.

Marvin Minsky and Seymour A. Papert. *Perceptrons: An Introduction to Computational Geometry*. MIT Press, Cambridge, Massachusetts, 1969.

Neel Nanda, Lawrence Chan, Tom Lieberum, Jess Smith, and Jacob Steinhardt. Progress measures for grokking via mechanistic interpretability, 2023. URL https://arxiv.org/abs/2301.05217.

Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Scott Johnston, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. In-context learning and induction heads, 2022. URL https://arxiv.org/abs/2209.11895.

OpenAI. Prompt used for "a proof of the cycle double cover conjecture". Technical note, July 2026. URL https://cdn.openai.com/pdf/04d1d1e4-bc75-476a-97cf-49055cd98d31/cdc_prompt.pdf.

Raj Pathak and Stefano Fabbri. Using goals in Codex. OpenAI Cookbook, May 2026. URL https://developers.openai.com/cookbook/examples/codex/using_goals_in_codex.

Ramamohan Paturi. On the degree of polynomials that approximate symmetric boolean functions. In *Proceedings of the Twenty-Fourth Annual ACM Symposium on Theory of Computing*, pages 468–474. ACM, 1992. doi: 10.1145/129712.129758. URL https://doi.org/10.1145/129712.129758.

Mateusz Piotrowski, Paul M. Riechers, Daniel Filan, and Adam S. Shai. Constrained belief updates explain geometric structures in transformer representations, 2025. URL https://arxiv.org/abs/2502.01954.

Marco Sälzer, Chris Köcher, Alexander Kozachinskiy, Georg Zetzsche, and Anthony Widjaja Lin. The counting power of transformers, 2026. URL https://arxiv.org/abs/2505.11199. Accepted at ICLR 2026.

Or David Shafran, Shaked Ronen, Omri Fahn, Shauli Ravfogel, Atticus Geiger, and Mor Geva. From directions to regions: Decomposing activations in language models via local geometry. In *Forty-third International Conference on Machine Learning*, 2026. URL https://openreview.net/forum?id=Tz7n7pX9c0.

George Tsoukalas, Anton Kovsharov, Sergey Shirobokov, Anja Surina, Moritz Firsching, Gergely Bérczi, Francisco J. R. Ruiz, Arun Suggala, Adam Zsolt Wagner, Eric Wieser, Lei Yu, Aja Huang, Miklós Z. Horváth, Andrew Ferraiuolo, Henryk Michalewski, Edward Lockhart, Codrut Grosu, Thomas Hubert, Matej Balog, Pushmeet Kohli, and Swarat Chaudhuri. Advancing mathematics research with ai-driven formal proof search, 2026. URL https://arxiv.org/abs/2605.22763.

UK AI Security Institute. Inspect AI: Framework for large language model evaluations. Open-source software, May 2024. URL https://github.com/UKGovernmentBEIS/inspect_ai.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008, 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.

Karthik Viswanathan, Yuri Gardinazzi, Giada Panerai, Alberto Cazzaniga, and Matteo Biagetti. The intrinsic dimension of prompts in internal representations of large language models. *Transactions on Machine Learning Research*, 2026. ISSN 2835-8856. URL https://openreview.net/forum?id=rBEgNAslpY.

Gail Weiss, Yoav Goldberg, and Eran Yahav. Thinking like transformers. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 11080–11090. PMLR, 2021. URL https://proceedings.mlr.press/v139/weiss21a.html.

Andy Yang and David Chiang. Counting like transformers: Compiling temporal counting logic into softmax transformers. In *Proceedings of the First Conference on Language Modeling*, 2024. URL https://openreview.net/forum?id=FmhPg4UJ9K.

Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan J. Prenger, and Anima Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems*, volume 36, pages 21573–21612, 2023. doi: 10.52202/075280-0944. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/4441469427094f8873d0fecb0c4e1cee-Abstract-Datasets_and_Benchmarks.html.

Penghao Yu, Haotian Jiang, Zeyu Bao, Ruoxi Yu, and Qianxiao Li. The effect of attention head count on transformer approximation, 2026. URL https://arxiv.org/abs/2510.06662. Accepted at ICLR 2026.

D Relation to Concurrent Work by Hsu and Xu

Overlapping results. In concurrent work closest to our setting, Hsu and Xu [2026] prove that an H-head attention layer with a degree-D polynomial-threshold readout representing f must satisfy $DH \geq \deg_{\pm}(f)$, and that this lower bound is exact for symmetric functions. At $D = 1$, their result agrees with our symmetric characterization in Theorem 5, giving the same minimum head count for nonconstant symmetric functions. We developed the symmetric characterization independently of the publicly released Hsu–Xu manuscript.

Differences in scope. Their model omits positional embeddings, whereas our fixed-query, linear-readout model allows arbitrary additive positional embeddings shared across heads. Theorem 5 therefore shows that positional information does not lower the required head count for symmetric functions. Beyond the symmetric setting, we additionally derive an exact linear-fractional normal form, prove the universal bound $H^*(f) \leq 2^n - 1$, and exhibit a nonsymmetric function with $\deg_{\pm}(f) = 2 < 3 \leq H^*(f)$, showing that threshold degree does not characterize head complexity. Additionally, we also make publicly available a Lean 4 formalization of the attention model and the main results, including the symmetric characterization, the linear-fractional normal form, and the nonsymmetric separation.

E Model Details

This section gives a self-contained specification of the model underlying Equations (1) and (2). Fix an input length n and a number of heads H. The model receives an n-bit input, applies a single multi-head self-attention update, and classifies the resulting representation at a dedicated query position. Figure 1 gives an overview of this computation.

Input sequence and representations. For $\mathbf{x} = (x_1, \ldots, x_n) \in \{0,1\}^n$, append a dedicated query token q to form the sequence $(x_1, \ldots, x_n, q)$. The query token is a distinguished vocabulary element, for which the symbol $=$ is one possible choice. Let $\mathcal{I} := [n] \cup \{q\}$ denote the set of sequence positions.

Choose a model dimension d_{model}, token embeddings $\mathbf{e}_0, \mathbf{e}_1, \mathbf{e}_q \in \mathbb{R}^{d_{\text{model}}}$, and positional embeddings $\mathbf{p}_1, \ldots, \mathbf{p}_n, \mathbf{p}_q \in \mathbb{R}^{d_{\text{model}}}$. Following the standard additive positional-encoding convention, the representation at each position is the sum of its token embedding and its positional embedding:

$$\mathbf{u}_j(\mathbf{x}) = \begin{cases} \mathbf{e}_{x_j} + \mathbf{p}_j, & j \in [n], \\ \mathbf{e}_q + \mathbf{p}_q, & j = q. \end{cases} \tag{7}$$

In each case, the first term identifies the token occupying the position and the second encodes the position itself. Since the token at the query position is always q, $\mathbf{u}_q$ is independent of $\mathbf{x}$. These token and positional embeddings are shared across heads; each head operates on the same embedded sequence $\{\mathbf{u}_j(\mathbf{x})\}_{j \in \mathcal{I}}$.

Single-head attention update. Choose a head dimension d_{head}. Each head $h \in [H]$ has its own query, key, and value projections

$$W_Q^{(h)}, W_K^{(h)}, W_V^{(h)} \in \mathbb{R}^{d_{\text{head}} \times d_{\text{model}}} \tag{8}$$

and output projection $W_O^{(h)} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{head}}}$. At the query position, head h assigns position $j \in \mathcal{I}$ the logit and normalized attention weight

$$\ell_j^{(h)}(\mathbf{x}) = \left\langle W_K^{(h)} \mathbf{u}_j(\mathbf{x}), W_Q^{(h)} \mathbf{u}_q \right\rangle, \tag{9}$$

$$a_j^{(h)}(\mathbf{x}) = \frac{\exp\left(\ell_j^{(h)}(\mathbf{x})\right)}{\sum_{r \in \mathcal{I}} \exp\left(\ell_r^{(h)}(\mathbf{x})\right)}. \tag{10}$$

Its contribution to the query position is

$$\mathbf{y}_q^{(h)}(\mathbf{x}) = W_O^{(h)} \sum_{j \in \mathcal{I}} a_j^{(h)}(\mathbf{x}) W_V^{(h)} \mathbf{u}_j(\mathbf{x}) \in \mathbb{R}^{d_{\text{model}}}. \tag{11}$$

Only the output at q is used for readout; outputs at other positions are not analyzed in this work.

Multi-head output and readout. The projected head outputs are added to the residual stream at the query position:

$$\mathbf{h}_q(\mathbf{x}) = \mathbf{u}_q + \sum_{h=1}^{H} \mathbf{y}_q^{(h)}(\mathbf{x}). \tag{12}$$

Given a readout vector $\mathbf{w} \in \mathbb{R}^{d_{\text{model}}}$ and threshold $\tau \in \mathbb{R}$, an H-head configuration exactly represents $f : \{0,1\}^n \to \{0,1\}$ if

$$\forall \mathbf{x} \in \{0,1\}^n, \qquad \big[f(\mathbf{x}) = 1 \quad \Longleftrightarrow \quad \langle \mathbf{w}, \mathbf{h}_q(\mathbf{x}) \rangle > \tau\big]. \tag{13}$$

The head complexity $H^*(f)$ is the minimum $H \in \mathbb{Z}_{\geq 0}$ over all choices of dimensions, embeddings, projections, and readout parameters that exactly represent f.

Conventions and scope. All parameters are real-valued, and d_{model} and d_{head} are unrestricted. The usual factor $1/\sqrt{d_{\text{head}}}$ in scaled dot-product attention is omitted because it can be absorbed into the query or key projections. Likewise, $W_O^{(h)}$ can be composed with $W_V^{(h)}$; we retain the output projections to distinguish the model and head dimensions. The residual $\mathbf{u}_q$ is independent of $\mathbf{x}$ and can therefore be absorbed into the readout threshold. Unrestricted width allows the shared embeddings to allocate orthogonal coordinate blocks selected by different heads.

The query token occurs last, so full attention and a standard causal mask expose it to the same set of positions. The architecture contains no MLP, layer normalization, dropout, or additional layers.

F Proofs of the Main Results

F.1 Linear-Fractional Normal Form

This appendix derives Definition 1 from Equation (1) and proves Theorem 1. Fix an H-head model with readout $(\mathbf{w}, \tau)$, and let $\mathbf{y}_q^{(h)}(\mathbf{x})$ denote the projected output of head h, that is, the h-th summand in Equation (1). Define its scalar contribution after readout by

$$g^{(h)}(\mathbf{x}) := \left\langle \mathbf{w}, \mathbf{y}_q^{(h)}(\mathbf{x}) \right\rangle, \qquad h \in [H], \tag{14}$$

Taking the inner product of Equation (1) with $\mathbf{w}$ then gives

$$\langle \mathbf{w}, \mathbf{h}_q(\mathbf{x}) \rangle = \langle \mathbf{w}, \mathbf{u}_q \rangle + \sum_{h=1}^{H} g^{(h)}(\mathbf{x}), \tag{15}$$

Since the first term is independent of $\mathbf{x}$, it can serve as the constant c in Definition 2. Thus, proving $L_{\text{frac}}(f) \leq H^*(f)$ reduces to showing that each $g^{(h)}$ is a linear-fractional atom.

From a head to an atom. For a head h, define

$$\mathbf{q}^{(h)} = W_Q^{(h)} \mathbf{u}_q, \qquad \Delta\mathbf{e} = \mathbf{e}_1 - \mathbf{e}_0. \tag{16}$$

The query vector $\mathbf{q}^{(h)}$ is independent of the input. Since $x_i \in \{0,1\}$, the representation at input position i can be written as

$$\mathbf{u}_i(\mathbf{x}) = \mathbf{e}_{x_i} + \mathbf{p}_i = \mathbf{e}_0 + \mathbf{p}_i + x_i \Delta\mathbf{e}. \tag{17}$$

Let $\ell_j^{(h)}(\mathbf{x})$ denote the attention logit at position j, and let $s_j^{(h)}(\mathbf{x})$ denote its scalar value after the value projection, output projection, and readout:

$$\ell_j^{(h)}(\mathbf{x}) = \left\langle W_K^{(h)} \mathbf{u}_j(\mathbf{x}), \mathbf{q}^{(h)} \right\rangle, \qquad s_j^{(h)}(\mathbf{x}) = \left\langle \mathbf{w}, W_O^{(h)} W_V^{(h)} \mathbf{u}_j(\mathbf{x}) \right\rangle. \tag{18}$$

Substituting Equation (17) into the logit gives

$$\ell_i^{(h)}(\mathbf{x}) = \left\langle W_K^{(h)}(\mathbf{e}_0 + \mathbf{p}_i), \mathbf{q}^{(h)} \right\rangle + x_i \left\langle W_K^{(h)} \Delta\mathbf{e}, \mathbf{q}^{(h)} \right\rangle, \tag{19}$$

$$\exp\Big(\ell_i^{(h)}(\mathbf{x})\Big) = \rho_i^{(h)} \big(\alpha^{(h)}\big)^{x_i}, \tag{20}$$

where

$$\rho_i^{(h)} = \exp\Big(\Big\langle W_K^{(h)}(\mathbf{e}_0 + \mathbf{p}_i), \mathbf{q}^{(h)} \Big\rangle\Big), \qquad \alpha^{(h)} = \exp\Big(\Big\langle W_K^{(h)} \Delta\mathbf{e}, \mathbf{q}^{(h)} \Big\rangle\Big). \tag{21}$$

Likewise, substituting Equation (17) into the scalar value gives

$$s_i^{(h)}(\mathbf{x}) = m_i^{(h)} + \beta^{(h)} x_i, \tag{22}$$

$$m_i^{(h)} = \Big\langle \mathbf{w}, W_O^{(h)} W_V^{(h)} (\mathbf{e}_0 + \mathbf{p}_i) \Big\rangle, \qquad \beta^{(h)} = \Big\langle \mathbf{w}, W_O^{(h)} W_V^{(h)} \Delta\mathbf{e} \Big\rangle. \tag{23}$$

Thus $\rho_i^{(h)}$ and $m_i^{(h)}$ are the unnormalized attention weight and the value of position i when $x_i = 0$, while $\alpha^{(h)}$ is the factor gained and $\beta^{(h)}$ the shift incurred when $x_i = 1$. The latter two depend only on $\Delta\mathbf{e}$, so they are the same at every position. At the query position, both quantities are input-independent. Set

$$\gamma^{(h)} = \exp\Big(\Big\langle W_K^{(h)} \mathbf{u}_q, \mathbf{q}^{(h)} \Big\rangle\Big), \qquad \eta^{(h)} = \gamma^{(h)} \Big\langle \mathbf{w}, W_O^{(h)} W_V^{(h)} \mathbf{u}_q \Big\rangle. \tag{24}$$

The scalar contribution of head h to the readout score is therefore

$$\begin{aligned} g^{(h)}(\mathbf{x}) &= \frac{\sum_{j\in\mathcal{I}} \exp\Big(\ell_j^{(h)}(\mathbf{x})\Big)\, s_j^{(h)}(\mathbf{x})}{\sum_{j\in\mathcal{I}} \exp\Big(\ell_j^{(h)}(\mathbf{x})\Big)} \\ &= \frac{\eta^{(h)} + \sum_{i=1}^n \rho_i^{(h)} \big(\alpha^{(h)}\big)^{x_i} \big(m_i^{(h)} + \beta^{(h)} x_i\big)}{\gamma^{(h)} + \sum_{i=1}^n \rho_i^{(h)} \big(\alpha^{(h)}\big)^{x_i}}. \end{aligned} \tag{25}$$

The exponential definitions imply $\rho_i^{(h)}, \alpha^{(h)}, \gamma^{(h)} > 0$, so Equation (25) is a linear-fractional atom in the sense of Definition 1. On the cube, $\alpha^{x_i} = 1 + (\alpha - 1)x_i$ and $x_i^2 = x_i$, so its numerator and denominator are affine functions of $\mathbf{x}$, and the denominator is positive. By Equation (15), the complete pre-threshold score is

$$\langle \mathbf{w}, \mathbf{h}_q(\mathbf{x}) \rangle - \tau = \underbrace{\langle \mathbf{w}, \mathbf{u}_q \rangle - \tau}_{c} + \sum_{h=1}^{H} g^{(h)}(\mathbf{x}). \tag{26}$$

Thus every H-head representation gives an H-atom representation, proving $L_{\mathrm{frac}}(f) \le H^*(f)$.

From atoms to heads. Conversely, consider prescribed atoms $\phi_1, \dots, \phi_H$, with their parameters indexed by h. Choose $d_{\mathrm{model}} \ge 4H$ and orthonormal vectors $\mathbf{q}^{(h)}, \mathbf{k}^{(h)}, \mathbf{v}^{(h)}, \mathbf{o}^{(h)} \in \mathbb{R}^{d_{\mathrm{model}}}$ for each h. Let $d_{\mathrm{head}} \ge 1$ be arbitrary, and fix a unit vector $\mathbf{r} \in \mathbb{R}^{d_{\mathrm{head}}}$. Choose the embeddings

$$\mathbf{e}_0 = 0, \qquad \mathbf{e}_1 = \sum_h \Big((\log \alpha^{(h)}) \mathbf{k}^{(h)} + \beta^{(h)} \mathbf{v}^{(h)} \Big), \tag{27}$$

$$\begin{aligned} \mathbf{e}_q &= \sum_h \left(\mathbf{q}^{(h)} + (\log \gamma^{(h)}) \mathbf{k}^{(h)} + \frac{\eta^{(h)}}{\gamma^{(h)}} \mathbf{v}^{(h)} \right), \\ \mathbf{p}_i &= \sum_h \Big((\log \rho_i^{(h)}) \mathbf{k}^{(h)} + m_i^{(h)} \mathbf{v}^{(h)} \Big), \qquad \mathbf{p}_q = 0. \end{aligned} \tag{28}$$

For each head h, define the linear maps on these directions by

$$W_Q^{(h)} \mathbf{q}^{(h)} = \mathbf{r}, \qquad W_K^{(h)} \mathbf{k}^{(h)} = \mathbf{r}, \qquad W_V^{(h)} \mathbf{v}^{(h)} = \mathbf{r}, \qquad W_O^{(h)} \mathbf{r} = \mathbf{o}^{(h)}, \tag{29}$$

extending each map by zero on the orthogonal complement of its displayed input direction. The logits at input position i and the query position are

$$\log \rho_i^{(h)} + x_i \log \alpha^{(h)} \qquad \text{and} \qquad \log \gamma^{(h)}, \tag{30}$$

and the corresponding scalar values in the $\mathbf{o}^{(h)}$ direction are $m_i^{(h)} + \beta^{(h)} x_i$ and $\eta^{(h)}/\gamma^{(h)}$. Hence head h writes $\phi_h(\mathbf{x})\mathbf{o}^{(h)}$. Reading in direction $\sum_h \mathbf{o}^{(h)}$ and absorbing the desired constant into the threshold realizes the sum of the atoms. The query residual is orthogonal to the readout. This proves $H^*(f) \le L_{\mathrm{frac}}(f)$, and the two inequalities establish the theorem.

F.2 One-Head Characterization

We prove Corollary 1.

Zero heads. When $H = 0$, Equation (1) reduces to $\mathbf{h}_q(\mathbf{x}) = \mathbf{u}_q$, which is independent of $\mathbf{x}$. Every zero-head classifier is therefore constant. Conversely, either constant function can be realized by placing the readout threshold above or below the fixed score $\langle \mathbf{w}, \mathbf{u}_q \rangle$. Hence $H^*(f) = 0$ if and only if f is constant.

At most one head implies linear threshold. Suppose $H^*(f) \leq 1$. The zero-head case is covered above. Otherwise, Theorem 1 gives a constant c and one atom $\phi = N/D$ such that $f(\mathbf{x}) = 1$ exactly when $c + \phi(\mathbf{x}) > 0$. The numerator N and denominator D are affine on the Boolean cube, and $D(\mathbf{x}) > 0$. Therefore

$$c + \phi(\mathbf{x}) = \frac{cD(\mathbf{x}) + N(\mathbf{x})}{D(\mathbf{x})} \tag{31}$$

has the same sign as the affine function $cD + N$. Thus f is a linear threshold function.

Linear threshold implies at most one head. Suppose $f(\mathbf{x}) = 1$ exactly when $\theta_0 + \sum_{i=1}^n \theta_i x_i > 0$. In Equation (4), take

$$\alpha = 2, \qquad \gamma = 1, \qquad \rho_i = 1, \qquad \beta = 0, \qquad m_i = \theta_i, \qquad \eta = \theta_0 - \sum_{i=1}^n \theta_i. \tag{32}$$

Since $2^{x_i} = 1 + x_i$ on the Boolean cube, the numerator of the resulting atom is

$$\theta_0 - \sum_{i=1}^n \theta_i + \sum_{i=1}^n 2^{x_i}\theta_i = \theta_0 + \sum_{i=1}^n \theta_i x_i. \tag{33}$$

Its denominator $1 + \sum_i 2^{x_i}$ is strictly positive, so the atom has the same sign as the given affine threshold function. Taking $c = 0$ in Definition 2 yields $L_{\mathrm{frac}}(f) \leq 1$, and Theorem 1 then gives $H^*(f) \leq 1$.

F.3 Threshold-Degree Lower Bound

We prove Theorem 2. On the Boolean cube, the numerator and denominator of every atom in Equation (4) are affine functions, because $x_i \in \{0, 1\}$ and $x_i^2 = x_i$. After moving the readout threshold into a constant, an H-head classifier therefore has a scalar score

$$S(\mathbf{x}) = c + \sum_{h=1}^H \frac{N_h(\mathbf{x})}{D_h(\mathbf{x})}, \qquad D_h(\mathbf{x}) > 0, \tag{34}$$

where N_h and D_h are affine. Since the domain is finite, the threshold can be perturbed so that S is strictly positive on inputs labelled one and strictly negative on inputs labelled zero. Multiplication by the positive common denominator preserves this sign pattern and gives

$$P(\mathbf{x}) = c \prod_{h=1}^H D_h(\mathbf{x}) + \sum_{h=1}^H N_h(\mathbf{x}) \prod_{g \neq h} D_g(\mathbf{x}). \tag{35}$$

Every summand has degree at most H, so P is a degree-at-most-H sign representation. Minimizing over H proves $\deg_{\pm}(f) \leq H^*(f)$.

F.4 Strict Separation from Threshold Degree

We prove Theorem 3 for the function in Equation (5). For its Boolean blocks $\mathbf{a}, \mathbf{b} \in \{0, 1\}^5$, introduce the sign coordinates

$$s_i = 1 - 2a_i, \qquad t_i = 1 - 2b_i, \qquad i \in [5], \tag{36}$$

so $\mathbf{s}, \mathbf{t} \in \{-1, +1\}^5$. Define the corresponding quadratic form on the sign cube by

$$\widetilde{Q}(\mathbf{s}, \mathbf{t}) = \left(\sum_{i=1}^5 s_i\right)\left(\sum_{j=1}^5 t_j\right) - 3\sum_{i=1}^5 s_i t_i. \tag{37}$$

Equation (5) then gives $Q(\mathbf{a}, \mathbf{b}) = \widetilde{Q}(\mathbf{s}, \mathbf{t})$. The coordinate change is an invertible affine map between the Boolean and sign cubes. Substitution in either direction does not increase polynomial degree, so threshold degree is unchanged; affine forms also remain affine.

Let A and B be the numbers of negative entries in $\mathbf{s}$ and $\mathbf{t}$, and let C count coordinates at which both are negative. Direct substitution gives

$$\widetilde{Q}(\mathbf{s}, \mathbf{t}) = 10 + 4(-A - B + AB - 3C) \equiv 2 \pmod 4. \tag{38}$$

Thus $\widetilde{Q}$ never vanishes, so the polynomial Q in Equation (5) is a strict degree-two sign representation of f_{10}. On the two-dimensional face of the sign cube

$$\mathbf{s} = (r, 1, 1, -1, -1), \qquad \mathbf{t} = (w, 1, -1, 1, -1), \qquad r, w \in \{-1, +1\}, \tag{39}$$

we have $\widetilde{Q}(\mathbf{s}, \mathbf{t}) = -2rw$. Under the inverse coordinate change, this is a two-dimensional face of the original Boolean cube whose sign pattern is XOR, so it cannot be sign-represented by an affine function. Consequently, $\deg_{\pm}(f_{10}) = 2$.

It remains to rule out two heads. If two heads represented f_{10}, Equation (35) would give a strict sign-representing polynomial

$$P = cD_1D_2 + N_1D_2 + N_2D_1 = (cD_1 + N_1)D_2 + N_2D_1. \tag{40}$$

Since N_1, N_2, D_1, D_2 are affine, this is a sum of two products of affine forms. Pull this polynomial back to the sign cube by setting

$$P^{\pm}(\mathbf{s}, \mathbf{t}) = P\left(\frac{\mathbf{1} - \mathbf{s}}{2}, \frac{\mathbf{1} - \mathbf{t}}{2}\right), \tag{41}$$

where $\mathbf{1}$ denotes the all-ones vector. Because the coordinate change is affine, the regrouping retains the form

$$P^{\pm} = L_1R_1 + L_2R_2, \tag{42}$$

where all four factors are affine in $(\mathbf{s}, \mathbf{t})$, and $P^{\pm}$ has the sign pattern of $\widetilde{Q}$. For a function g on the two signed blocks, define its mixed antipodal difference

$$\Delta g(\mathbf{s}, \mathbf{t}) = g(\mathbf{s}, \mathbf{t}) - g(\mathbf{s}, -\mathbf{t}) - g(-\mathbf{s}, \mathbf{t}) + g(-\mathbf{s}, -\mathbf{t}). \tag{43}$$

Writing an affine form as $L(\mathbf{s}, \mathbf{t}) = \ell_0 + \mathbf{c}_L \cdot \mathbf{s} + \mathbf{d}_L \cdot \mathbf{t}$ gives

$$\Delta(LR)(\mathbf{s}, \mathbf{t}) = 4\big[(\mathbf{c}_L \cdot \mathbf{s})(\mathbf{d}_R \cdot \mathbf{t}) + (\mathbf{c}_R \cdot \mathbf{s})(\mathbf{d}_L \cdot \mathbf{t})\big]. \tag{44}$$

Each product L_hR_h therefore contributes two outer products, $4\mathbf{c}_{L_h}\mathbf{d}_{R_h}^{\top}$ and $4\mathbf{c}_{R_h}\mathbf{d}_{L_h}^{\top}$, to the mixed coefficient matrix. Each has rank at most one. Thus $\Delta P^{\pm}(\mathbf{s}, \mathbf{t}) = \mathbf{s}^{\top}K\mathbf{t}$ for a matrix K with $\operatorname{rank}(K) \le 4$.

For $j \in [5]$, let $\mathbf{z}^{(j)}$ have entry -1 in coordinate j and $+1$ elsewhere. Equation (37) gives

$$\widetilde{Q}(\mathbf{s}, \mathbf{z}^{(j)}) = 6s_j, \tag{45}$$

and negating either whole block reverses its sign. Set $Z = [\mathbf{z}^{(1)} \ \cdots \ \mathbf{z}^{(5)}]$ and $M = KZ$. The jth column of M is the coefficient vector of the slice $\Delta P^{\pm}(\mathbf{s}, \mathbf{z}^{(j)}) = \sum_i M_{ij}s_i$, and $\operatorname{rank}(M) \le \operatorname{rank}(K) \le 4$. Since $P^{\pm}$ has the sign pattern of $\widetilde{Q}$, all four terms in this difference have sign s_j, giving

$$s_j \sum_i M_{ij}s_i > 0 \qquad \text{for every } \mathbf{s} \in \{-1, +1\}^5. \tag{46}$$

Fixing $s_j = 1$ and choosing every other sign to oppose M_{ij} yields

$$M_{jj} > \sum_{i \neq j} |M_{ij}|. \tag{47}$$

Thus M is strictly column diagonally dominant and therefore invertible, so it has rank five. This contradicts the bound $\operatorname{rank}(M) \le 4$ established above. Two heads cannot represent f_{10}, proving $3 \le H^*(f_{10})$ and completing Theorem 3.

F.5 Weighted-Sum Interpolation

We prove Theorem 4. Set $M = |\operatorname{Im}(t_\lambda)|$. The claim is immediate when $M = 1$, so assume $M \geq 2$. Let $\Lambda = \sum_i \lambda_i$ and enumerate the image of t_λ as $0 = \tau_0 < \tau_1 < \cdots < \tau_{M-1}$.

Choose distinct $\alpha_1, \ldots, \alpha_{M-1} > 1$. For each j, take the atom in Equation (4) with $\gamma = 1$, $\rho_i = \lambda_i$, $\alpha = \alpha_j$, $\eta = m_i = 0$, and $\beta = 1$. On the Boolean cube, this gives

$$\phi_j(\mathbf{x}) = \frac{\alpha_j t_\lambda(\mathbf{x})}{1 + \Lambda + (\alpha_j - 1) t_\lambda(\mathbf{x})} = g_j(t_\lambda(\mathbf{x})). \tag{48}$$

It remains to show that $1, g_1, \ldots, g_{M-1}$ form a basis for functions on $\operatorname{Im}(t_\lambda)$. Suppose a linear combination of these functions vanishes on the image. Evaluation at $\tau_0 = 0$ eliminates its constant coefficient. Dividing the remaining equations by each $\tau_m > 0$ and setting $r_j = (1 + \Lambda)/(\alpha_j - 1)$ reduces them to $\sum_j d_j/(\tau_m + r_j) = 0$ for $m = 1, \ldots, M - 1$. After multiplication by $\prod_j (s + r_j)$, the corresponding rational function has a numerator of degree at most $M - 2$ with $M - 1$ distinct roots. It is therefore identically zero. Evaluating at each pole $-r_j$ shows that every d_j, and hence every original coefficient, vanishes. The M functions are linearly independent on an M-point set, so they form a basis.

Interpolating the target signs gives a score $c + \sum_{j=1}^{M-1} a_j g_j(t_\lambda(\mathbf{x}))$. Each scaled term is an atom: replace $\beta = 1$ by $\beta = a_j$ in the construction above. The normal form in Theorem 1 therefore gives an $(M - 1)$-head representation. Taking $\lambda_i = 2^{i-1}$ makes t_λ injective on the Boolean cube, so $M = 2^n$ and $H^*(f) \leq 2^n - 1$ for every Boolean function.

F.6 Exact Symmetric Characterization

We prove Theorem 5. Write $\sigma_k = +1$ when $F(k) = 1$ and $\sigma_k = -1$ otherwise, and let

$$T = \{k \in [n] : \sigma_{k-1} \neq \sigma_k\}, \qquad |T| = C(F). \tag{49}$$

The polynomial

$$P(z) = \sigma_0 \prod_{k \in T} \left(k - \frac{1}{2} - z \right) \tag{50}$$

has sign σ_k at every integer $k \in \{0, \ldots, n\}$ and degree $C(F)$. Thus $P(|\mathbf{x}|)$ sign-represents f.

For the reverse threshold-degree inequality, take any degree-d polynomial sign-representing f, multilinearize it on the Boolean cube, and average it over all coordinate permutations. The averaged polynomial remains a strict sign representation because f is symmetric. Every symmetric multilinear polynomial of degree at most d, evaluated on a Boolean input of weight k, is a univariate polynomial $R(k)$ of degree at most d: the degree-m elementary symmetric polynomial evaluates to $\binom{k}{m}$. Whenever F changes between $k - 1$ and k, the values $R(k - 1)$ and $R(k)$ have opposite signs, so R has a root in $(k - 1, k)$. These intervals are disjoint, giving $d \geq C(F)$. Hence

$$\deg_\pm(f) = C(F), \tag{51}$$

and Theorem 2 gives $H^*(f) \geq C(F)$.

For the upper bound, assume $C = C(F) > 0$ and choose distinct $r_1, \ldots, r_C > 0$. Set $B(z) = \prod_{h=1}^{C} (z + r_h)$. Since $B(k) > 0$ at every Hamming weight, P/B has the target sign pattern. Partial fractions give

$$\frac{P(z)}{B(z)} = c + \sum_{h=1}^{C} \frac{d_h}{z + r_h}. \tag{52}$$

Each shifted reciprocal is realized by one atom. Indeed, for fixed $r > 0$ and $d \in \mathbb{R}$, choose $\alpha > 1 + n/r$ in Equation (4), set

$$\rho_i = 1, \qquad \gamma = (\alpha - 1)r - n, \qquad \eta = (\alpha - 1)d, \qquad m_i = \beta = 0. \tag{53}$$

Then $\gamma > 0$ and the atom reduces exactly to $d/(|\mathbf{x}| + r)$. Equation (52) is therefore realized with C heads. If $C = 0$, f is constant and needs no heads. This proves $H^*(f) \leq C(F)$ and completes the equality.

G Initial Prompt and Repository README of the Autonomous Run

G.1 Initial prompt

The single user message that started the theorem-generation run, reproduced from the retained rollout (25 August 2026, 03:48:34 UTC). Typos are as in the original. The counterexample search was opened on 13 July 2026 with the one-line goal *find an explicit constructive example when* $\deg_{\pm}(f)$ *not equal to* $H^*(f)$; the retained rollout covers that single turn.

/goal Answer the main question posed in the `README.md` file. Here are some additional guidelines that should help you get to the best possible answer:

- Assume for purposes of this task that a complete affirmative result exists. Partial progress does not count unless it implies exactly the resolution above. In particular, proofs for very narrowly restricted boolean function classes, very weak upper or lower bounds, and trivial or simple enough cases are insufficient.
- Begin with a genuinely diverse portfolio of approaches. Explore substantially different formulations, invariants, reductions, algebraic or geometric viewpoints, extremal arguments, and computational sanity checks.
- Maintain an explicit registry of approach families. Group your experments by the mathematical idea they are using, not by superficial wording. If many attempts converge to one family, redirect some of them toward underexplored formulations.
- Do not allow one approach to dominate merely because it gives elegant reductions. A route that ends at a lemma equivalent in strength to the original conjecture is not close to completion unless it supplies a genuinely new proof of that lemma.
- When an approach stalls at a theorem-strength missing lemma, mark that route as blocked. Only continue working on it if someone proposes a materially new mechanism, invariant, or construction.
- Keep several incompatible proof routes alive through multiple rounds. Cross-pollinate ideas only after you have developed them far enough to expose their real strengths and gaps.
- Require results to container concrete theorems, constructions, equations, or counterexamples to previously proposed such. Reject status reports, vague optimism, and claims that an unproved global compatibility statement is "routine."
- You should repeatedly synthesize, challenge, redirect, and launch new rounds. Do not stop after the first wave fails. Produce a complete proof if one survives audit; otherwise report only the strongest rigorously proved derivation and its exact remaining gap. Do not return merely because current approaches fail or you report theorem-strength gaps. Continue launching new rounds, reopening blocked approaches only when there is a genuinely new mechanism, and searching for fresh formulations. Return only when a complete affirmative proof has been found and survives adversarial audit. Do not return a reduction, partial result, isolated missing lemma, "best effort" summary, or explanation of why the problem is difficult. Spend at least 8 hours on this before even thinking of returning or giving up. Public search may not be used for ordinary mathematical background or standard named theorems, neither to search for a solution to this exact conjecture or benchmark.

G.2 Repository README at the start of the run

The prompt refers to the main question posed in the repository's `README.md`. At the start of the run the repository contained only this file, `problem_statement.md`, and `model.md`; the agent rewrote the README during the run, so the version below is the one it was given, reproduced verbatim.

A from-scratch attack on the head-complexity problem: what is the minimum number of attention heads $H^*(f)$ that a single-layer, attention-only transformer with a linear readout needs to compute a Boolean function f?

This repo deliberately starts with only the problem definition. No prior results, proofs, or surveys are carried over.

- `problem_statement.md`: the problem, the central quantity $H^*(f)$, and the core questions.
- `model.md`: the precise architecture (embeddings, attention heads, residual stream, readout, masking convention, deliberate simplifications).

G.3 Problem statement at the start of the run

Reproduced verbatim from `problem_statement.md`, which did not change during the run.

Goal We want to characterize the minimum number of attention heads required to compute a Boolean function

$$f : \{0,1\}^n \to \{0,1\}$$

in a **single-layer attention-only transformer** with a **linear readout** from a designated query token. The motivating example is that, in the 2-bit setting, a single head can linearly realize functions such as AND and OR, but cannot realize XOR, while two heads suffice. This suggests that the number of heads may define a meaningful complexity measure for Boolean functions.

Model We fix the following architecture.

- **Input:** a sequence consisting of n input bits and one dedicated query token `=`.
- **Embedding:** token embeddings plus positional embeddings.
- **Computation:** a *single self-attention layer* with H parallel heads and no MLP.
- **Output:** the residual stream at the query token after the attention update.
- **Readout:** a linear probe (equivalently, a thresholded affine functional) applied to that query residual.

A Boolean function f is said to be **computable with H heads** if there exists a choice of embeddings, attention parameters, and linear readout such that the resulting classifier equals f on all inputs in $\{0,1\}^n$.

Main Quantity Define

$$H^*(f) := \min \{H : f \text{ is computable with } H \text{ heads in the above model}\}.$$

Our central problem is to understand $H^*(f)$ as a function of f.

Core Questions

1. **Exact characterization.** Can $H^*(f)$ be expressed, exactly or approximately, in terms of a known invariant of f?
2. **Lower bounds.** What techniques prove that a function requires at least H heads?
3. **Upper bounds.** Can we constructively realize broad classes of Boolean functions with few heads?
4. **Comparison to classical complexity measures.** How does $H^*(f)$ relate to:
   - threshold degree,
   - rational degree,
   - threshold density / sparsity,
   - circuit depth / threshold-circuit complexity,
   - Fourier or spectral complexity measures?
5. **Natural families.** What is $H^*(f)$ for standard functions such as:
   - AND, OR,
   - parity / XOR,
   - majority,
   - exact-count / symmetric functions,
   - address / indexing functions,
   - intersections of threshold functions?

Desired Outcome We would like to identify either:

- an invariant $I(f)$ such that $H^*(f)$ is controlled by $I(f)$ (ideally $H^*(f) \asymp I(f)$ on broad classes), or
- explicit function families for which the head complexity exhibits qualitatively new behavior compared to classical measures.

In short:

> **What is the right notion of Boolean-function complexity captured by the minimum number of attention heads in one-layer attention with a linear readout?**

G.4 Model specification at the start of the run

Reproduced verbatim from `model.md`, which did not change during the run.

Purpose This file fixes the mathematical model used throughout the project.
We study Boolean functions

$$f : \{0,1\}^n \to \{0,1\}$$

computed by a one-layer attention-only architecture with a linear readout from a designated query token.

Input Sequence For an input

$$x = (x_1, \ldots, x_n) \in \{0,1\}^n,$$

the model sees the sequence

$$(x_1, \ldots, x_n, =).$$

The final symbol = is a distinguished query token. We only read out the residual stream at that token.

Embeddings Fix a model dimension $d_{\text{model}} \geq 1$ and a head dimension $d_{\text{head}} \geq 1$. Choose token embeddings

$$e_0, e_1, e_= \in \mathbb{R}^{d_{\text{model}}}$$

and positional embeddings

$$p_1, \ldots, p_n, p_= \in \mathbb{R}^{d_{\text{model}}}.$$

For an input x, define the embedded vectors

$$u_i(x) := e_{x_i} + p_i \qquad \text{for } 1 \leq i \leq n,$$

and

$$u_=(x) := e_= + p_=.$$

Note that $u_=(x)$ is constant as a function of x.

One Attention Head For each head $h \in \{1, \ldots, H\}$, choose matrices

$$W_Q^{(h)}, W_K^{(h)}, W_V^{(h)} \in \mathbb{R}^{d_{\text{head}} \times d_{\text{model}}}.$$

Also choose output-projection blocks

$$W_O^{(h)} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{head}}}.$$

Equivalently, one may concatenate these blocks into a single matrix

$$W_O = \begin{bmatrix} W_O^{(1)} & \cdots & W_O^{(H)} \end{bmatrix} \in \mathbb{R}^{d_{\text{model}} \times H d_{\text{head}}}.$$

For each position

$$j \in \{1, \ldots, n, =\},$$

define

$$q^{(h)}(x) := W_Q^{(h)} u_=(x) \in \mathbb{R}^{d_{\text{head}}},$$

$$k_j^{(h)}(x) := W_K^{(h)} u_j(x) \in \mathbb{R}^{d_{\text{head}}},$$

and

$$v_j^{(h)}(x) := W_V^{(h)} u_j(x) \in \mathbb{R}^{d_{\text{head}}}.$$

The query-token logit is

$$\ell_j^{(h)}(x) := \big(q^{(h)}(x)\big)^\top k_j^{(h)}(x).$$

The attention weights are the softmax probabilities

$$\alpha_j^{(h)}(x) := \frac{\exp\big(\ell_j^{(h)}(x)\big)}{\sum_{k \in \{1, \ldots, n, =\}} \exp\big(\ell_k^{(h)}(x)\big)}.$$

The unprojected output of head h at the query position is

$$\widetilde{y}^{(h)}(x) := \sum_{j \in \{1, \ldots, n, =\}} \alpha_j^{(h)}(x) v_j^{(h)}(x) \in \mathbb{R}^{d_{\text{head}}}.$$

The projected contribution of head h to the residual stream is

$$y^{(h)}(x) := W_O^{(h)} \widetilde{y}^{(h)}(x) \in \mathbb{R}^{d_{\text{model}}}.$$

Residual Stream At The Query Token With H parallel heads, the residual stream at the query token after the attention layer is

$$r(x) := u_=(x) + \sum_{h=1}^{H} y^{(h)}(x).$$

This is the only representation used by the final classifier.

Readout Choose a readout vector

$$w \in \mathbb{R}^{d_{\text{model}}}$$

and a threshold

$$\tau \in \mathbb{R}.$$

The classifier outputs

$$f(x) = 1 \qquad \Longleftrightarrow \qquad w^\top r(x) > \tau.$$

Equivalently, one may write an affine score

$$S(x) := w^\top r(x) - \tau$$

and classify by the rule

$$f(x) = 1 \qquad \Longleftrightarrow \qquad S(x) > 0.$$

Computability And Head Complexity A Boolean function f is **computable with H heads** if there exist:

- a model dimension d_{model},
- a head dimension d_{head},
- token and positional embeddings,
- attention parameters $\{W_Q^{(h)}, W_K^{(h)}, W_V^{(h)}, W_O^{(h)}\}_{h=1}^{H}$,
- a readout vector w and threshold τ,

such that the resulting classifier agrees with f on every input in $\{0,1\}^n$.
We then define

$$H^*(f) := \min \{H : f \text{ is computable with } H \text{ heads in the above model}\}.$$

Masking Convention The formulas above sum over all input positions and the query position itself. For the current project, this is equivalent to the usual causal-mask convention, because:

1. the query token = is placed last in the sequence,
2. only the residual stream at that final token is ever read out.

So a standard causal mask would still allow the query token to attend to every input bit and to itself. None of the current results depend on choosing between these two conventions.

Deliberate Simplifications The model in this project deliberately excludes several features of a full transformer block:

- no MLP,
- no layer normalization,
- no dropout,
- no multi-layer stacking,
- no extra nonlinearity after the attention update.

The only nonlinearity in the model is the softmax inside each attention head, followed by the final thresholding in the readout.

Notes On Conventions

1. Any constant scaling factor in the logits, such as $1/\sqrt{d_{\text{head}}}$, can be absorbed into the choice of $W_Q^{(h)}$ or $W_K^{(h)}$, so it is omitted from the specification.
2. Although W_O is now part of the formal model, many lower-bound arguments can compose the final linear readout with $W_O^{(h)}$ and thereby reduce each head again to a scalar contribution.
3. Because $u_=(x)$ is constant in x, any contribution from the query token's skip connection can always be absorbed into the readout threshold when proving lower or upper bounds.